



The Workers Caff: cheat sheet

Build with Cloudflare London · 29 September · 15:00 to 17:00 · prizes for the best builds at 17:00

Starter repo	github.com/GoncaloLeitao/workers-caff-hackathon
Your caff	<code>https://workers-caff.<you>.workers.dev</code> (Wrangler prints it)
Its pages	<code>/</code> dashboard · <code>/chat</code> Sid · <code>/mcp</code> MCP server · <code>/api/state</code> JSON
AI Playground	playground.ai.cloudflare.com/models
Full guide	<code>docs/guide.md</code> in the repo, with answers for every step



Scan for the starter repo

1 Get going

```
git clone https://github.com/GoncaloLeitao/workers-caff-hackathon.git
cd workers-caff-hackathon
npm install
npx wrangler login
npm run deploy
```

You need a free Cloudflare account, Node.js 22+ and Git. New accounts pick a `workers.dev` subdomain on the first deploy: any name works.

No Node here? Use **Deploy to Cloudflare** in the README, then GitHub's web editor (press `.` on your repo). Each commit redeploys.

2 The checkpoints

1 Open the caff

BY 15:35

1. Deploy, then open your Worker's URL: that's your dashboard.
2. In AI Playground open **Custom MCP** → **Add server** and paste your `/mcp` URL.
3. Ask it: *what's on the menu at the caff?*

Missions: Doors open · Hello, MCP · First tool call

2 Build the MCP server

BY 16:05

1. In `src/mcp.ts` finish TODO 1 to 4: `place_order`, `list_orders`, `update_order_status`, `restock_item`.
2. Deploy after each one, then **Custom MCP** → **Tools** → **Refresh**.
3. Try: *table 4 wants two bacon butties and a tea.*

Missions: Order up · Service! · Stock take

`skip:mcp`

3 Put Sid to work

BY 16:30

1. In `src/agent.ts` uncomment `this.addMcpServer(...)` and `tools:` `this.mcp.getAITools()`.
2. Deploy and open `/chat`. The header should show your tools.
3. Give Sid an order and watch the dashboard.

Missions: Agent on shift · Takes an order · Full service

`skip:agent`

4 Stretch

TILL 17:00

1. Refunds that a human approves on the dashboard.
2. A scheduled stock check with `this.scheduleEvery(...)`.
3. Memory for regulars, table hopping, a tool of your own, auth on `/mcp`. Or your own idea: `docs/ideas.md`.

Bonus: Off menu · Manager's say-so · Clockwork · Table hopping

`skip:stretch`

3 The code you'll write

```
// src/mcp.ts: a tool is a name, a description for the model, an input schema and a handler
server.registerTool(
  "restock_item",
  {
    description: "Add portions to a menu item's stock, e.g. after a delivery. Use item ids from get_menu.",
    inputSchema: z.object({
      itemId: z.string().describe("Menu item id from get_menu"),
      quantity: z.number().int().min(1).max(50).describe("How many portions to add")
    })
  },
  async ({ itemId, quantity }) => asText(await caff.restockItem(itemId, quantity))
);

// What caff can do. Prices are in pence, tables are 1 to 12, orders only move forward.
caff.getMenu()                caff.placeOrder({ table, items: [{ itemId, qty }], note })
caff.listOrders({ status, table })  caff.updateOrderStatus(orderId, "cooking" | "ready" | "served" | "cancelled")
caff.restockItem(itemId, quantity)  caff.requestRefund(orderId, reason)    caff.checkApproval(approvalId)

// src/agent.ts, TODO 1: connect Sid to your MCP server
await this.addMcpServer("caff", `${origin}/mcp`, {
  transport: { type: "streamable-http", headers: { "x-caff-client": "agent", "x-caff-origin": origin } }
});

// TODO 2, inside generateText({ ... }): give the model every tool your MCP server offers
tools: this.mcp.getAITools(),
```

`caff` throws readable errors (sold out, unknown item). The MCP SDK passes them to the model as tool errors, so you can skip the `try / catch`.

4 Commands

<code>npm run dev</code>	Local caff at <code>http://localhost:8787</code> . Test <code>/mcp</code> with MCP Inspector: AI Playground can't reach your laptop
<code>npm run deploy</code>	Put it live
<code>npm run smoke -- <url> --write --agent</code>	Check the API, your MCP tools and Sid in one go
<code>npx wrangler tail</code>	Live logs from your deployed Worker
<code>npx @modelcontextprotocol/inspector</code>	MCP Inspector: connect over Streamable HTTP and call tools by hand
<code>npm run skip:mcp · skip:agent · skip:all · skip:stretch</code>	Drop in a finished version. Yours is saved as <code>src/*.ts.bak</code>

5 If it breaks

Sid says he isn't connected	Finish both TODOs in <code>src/agent.ts</code> and deploy again
New tools missing in AI Playground	Custom MCP → Tools → Refresh
"Couldn't use your MCP server"	Something in <code>src/mcp.ts</code> throws, often a tool registered twice. <code>npm run smoke -- <url></code> shows the error
Error 1042	Keep <code>global_fetch_strictly_public</code> in <code>wrangler.jsonc</code> : Sid needs it to call your Worker's public URL
Free Workers AI allowance used up	Set <code>vars.MODEL</code> to <code>"@cf/zai-org/glm-4.7-flash"</code> in <code>wrangler.jsonc</code> and deploy